



Figure 3: Connected nodes and MQTT configuration

update the data. Topics and devices can also be brought in under organisations for better collaborative applications, where applicable public topics can be kept under suitable organisations. Refer to the API docs once, for the available REST APIs to access real-time and historical data.

Create a new flow, drag the SenseHAT input node (or simulator node) from the *Raspberry Pi* category and connect it to a MQTT out node as well as to a debug node, as shown in Figure 3, via trigger and JSON nodes. Configure the SenseHAT node to generate only environment events by deselecting motion and joystick events. By default, this node emits the `msg.payload` for certain periodic intervals with properties like temperature, humidity and pressure; and message topics such as environment, as per the selection. Since the data rate is very high, add a trigger node in between, with the configuration of *Send nothing, then wait for 10 seconds and then send the original msg.payload*. You may add a JSON node from the function category after the trigger node to publish data in the JSON format instead of the JavaScript object.

Configure MQTT output node with the following parameters. Under the *Connection* tab, fill `mqtt.opensensors.io` as the server host name, 1883 as the port number, and the device ID generated earlier as the client ID. Under the *Security* tab, fill in the user name and the password generated on device creation, and fill the topic name as `/users/xxxx/rpisense/weather` by replacing `xxxx` with your user name.

Simulator node works well with the same steps, except that you keep changing the sensor values manually under the UI tab, i.e., `localhost:1880/sensehat-simulator`.

Hit the deploy option and monitor the sensor values locally under the *Debug* tab, as well as published messages in the *Topics* section of the OpenSensors portal. You can also monitor these messages with *MQTT subscribe* from other devices with a similar configuration and topic, as this platform supports only one MQTT connection per device.

Local dashboard

You can also build a simple dashboard for local access, apart from publishing to the



Figure 4: Local dashboard

cloud. For this, connect the SenseHAT node to a function node, increase the number of outputs to three and fill the following code:

```
var msg1={}, msg2={}, msg3={};
msg1.payload=msg.payload.temperature;
msg2.payload=msg.payload.humidity;
msg3.payload=msg.payload.pressure;
return [ msg1,msg2,msg3 ];
```

Now, each output connector of this node can be connected to suitable dashboard nodes like gauge, chart, text, etc. Configure the dashboard nodes with *RaspberryPi* as the group name, *SenseHAT* as the tab name and give a suitable label and value range. Hit *Deploy* and open the link `hostname:1880/ui` to see the sensor values locally, as shown in Figure 4.

Note: The trigger node added earlier can only send the oldest or latest message by skipping the remaining ones. To get some consolidation like the mean, minimum and maximum of all the payloads, install the *node-red-contrib-aggregator* add-on, and add an aggregator node instead of a trigger node to see the desired values with applied statistics.

Writing custom nodes

It's time to prove that you can also contribute to Node-RED with your own additions. Any Node.js code can be easily wrapped into a custom node with a HTML frontend. For a simple example, let's try to make a node for retrieving data published to the topic in the above section. For this, real-time APIs are offered, which open a Server Side Events (SSE) stream for a given topic, and an API key is needed (if the topic is not public), which is generated in the *Profile* section. Given below is the console based Node.js code to watch a public topic with SSE client support.

```
var EventSource = require('eventsources');
var url=require('url');
var topic="/users/xxxx/rpisense/weather";
var urlOpts=url.parse("https://realtime.opensensors.io/v1/public/events/topics/"+topic);
var eventUrl = new EventSource(urlOpts);
eventUrl.onmessage = function(event) {
    console.log(event.data);
}
```

Let's follow the steps given below to wrap the above code into a Node-RED node.

1. Create a new directory `node-red-contrib-osfydemo` under `NODE_PATH`, i.e., `<topdir-of-nodejs>/lib/node_modules`.
2. From `NODE_PATH/node-red/nodes`, copy `99-sample.js.demo`, `99-sample.html.demo` as `opensorsensors.js`, `opensorsensors.html` in a sub-directory with name `opensorsensors` under `node-red-contrib-osfydemo`